

OpenClaw Integration Guide

Radware Agentic AI Protection

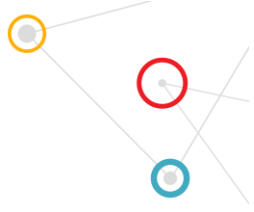
In-path and out-of-path deployment options

Written by: Sean Ramati

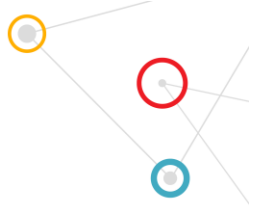
Version 1.1

June 29, 2026

NPM package: `openclaw-radware-agentic-protection@0.2.0`



For implementation details, use the published GitHub repository and NPM package.



Solution Overview

OpenClaw is an agent and gateway environment that can route user requests to LLM providers and execute tools or workflow actions through plugins. This integration supports two Radware Agentic AI Protection deployment options for OpenClaw: in-path provider routing or out-of-path explicit API enforcement.

The published solution can be used in a new OpenClaw deployment or added to an existing OpenClaw gateway without replacing existing agents, channels, tools, or provider configuration.

What We Published

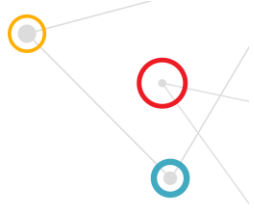
- GitHub repository with the customer guide, setup examples, validation scripts, and troubleshooting notes.
- NPM package that provides a single interactive radware-openclaw-setup wizard, installs the OpenClaw out-of-path plugin when needed, and preserves secrets outside openclaw.json.
- Validation evidence for AI Guardrails, Behavioral protection, Report Only behavior, and fail-open / fail-close handling.

GitHub: github.com/rdwr-seanr/openclaw-radware-agentic-protection

NPM: npmjs.com/package/openclaw-radware-agentic-protection

Guiding Recommendation

Choose exactly one integration path for a given OpenClaw deployment. Use in-path when Radware should sit inline on OpenClaw provider traffic. Use out-of-path when OpenClaw should call Radware explicitly from OpenClaw lifecycle hooks. New setup-generated out-of-path configs enable prompt, response, and tool-stage checks. Do not configure in-path and out-of-path together in the same OpenClaw environment.



OpenClaw Protection Model

In-path / Inline Protection

In-path protection routes OpenClaw provider traffic through Radware using an OpenAI-compatible provider configuration. OpenClaw uses the Radware in-path API key as the model-provider API key and sends prompt, response, and tool-call context to the Radware in-path base URL when that context is part of the provider request.

- Best fit: inline enforcement when Radware should be in the provider path for AI Guardrails and Behavioral / Agentic Protection.
- Credential rule: the Radware in-path key is used in the Radware provider entry. The customer OpenAI key is not placed in that entry.
- Operational behavior: in-path is fail-close by default because model traffic is routed through Radware.

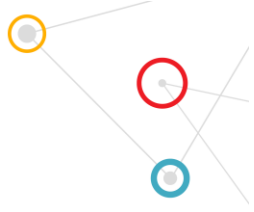
Out-of-path Explicit API Protection

Out-of-path protection keeps the customer LLM provider in place and adds explicit Radware checks from OpenClaw lifecycle hooks. New setup-generated configs enable prompt-stage, response-stage, and tool-stage enforcement. Existing configs without stage settings remain tool-stage only until upgraded intentionally.

- Best fit: explicit enforcement when the customer wants to keep the normal LLM provider while Radware checks prompt, response, and tool/action stages.
- Credential rule: the plugin uses the Radware out-of-path API key. The customer LLM provider key and endpoint stay in the customer's existing OpenClaw provider and are not collected by the Radware setup wizard.
- Portal behavior: Block and Report stops the protected stage; Report Only allows the stage and records the finding in Radware.

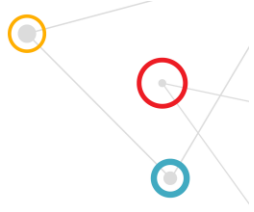
Credential Summary

In-path requires a Radware in-path API key. Out-of-path requires a Radware out-of-path API key and the customer LLM provider remains configured separately. The setup wizard writes keys to a chmod 600 runtime env file and never stores secrets in openclaw.json.



Provider Endpoint Notes

- In-path: the Radware base URL path must match the provider configured in the Radware portal. OpenAI is validated with <https://api.agentic.radwarecto.com/v1/openai>.
- Gemini in-path: direct Gemini OpenAI-compatible API worked at <https://generativelanguage.googleapis.com/v1beta/openai> with model gemini-2.5-flash, but tested Radware paths such as /v1/gemini and /v1/google did not produce a valid proxied Gemini result. Keep Gemini in-path as portal/Radware review until the correct path is confirmed.
- NVIDIA/Nemotron direct endpoint: <https://integrate.api.nvidia.com/v1> worked with model nvidia/nemotron-3-nano-30b-a3b. Test a dedicated Radware in-path deployment before documenting NVIDIA in-path as supported.
- Out-of-path: OpenClaw continues using the customer's existing LLM provider. The Radware plugin sends only the model identifier as ModelToUse for context.



Customer Rollout

Customer Installation

Prerequisite: OpenClaw is already installed, onboarded, and has an existing openclaw.json. Run commands as the same OS user that owns the OpenClaw config and runs the gateway.

1. Create the selected Radware protection in the portal and copy the Radware API key.
2. Run one command: `npx -y -p openclaw-radware-agentic-protection@latest radware-openclaw-setup`
3. Choose in-path or out-of-path and paste the Radware values when prompted. For out-of-path, keep the customer LLM provider already configured in OpenClaw.
4. Load the generated env file, then start OpenClaw: `set -a; . ~/.openclaw/radware.env; set +a; openclaw gateway run --force`
5. If setup fails, use the printed diagnostic-log path. For out-of-path runtime failures, review `~/.openclaw/logs/radware-agentic-runtime.jsonl` and the OpenClaw gateway log.

Easy Customer Commands

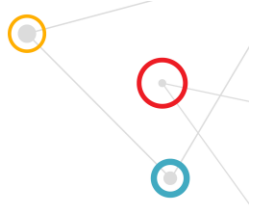
```
npx -y -p openclaw-radware-agentic-protection@latest radware-openclaw-setup
openclaw plugins install npm:openclaw-radware-agentic-protection@latest
set -a; . ~/.openclaw/radware.env; set +a; openclaw gateway run --force
```

Package Version Guidance

- Use `openclaw-radware-agentic-protection@latest` in customer-facing commands.
- Version 0.2.0 is the customer release version for the interactive wizard, full-stage out-of-path enforcement, sanitized diagnostics, and strict one-path-per-OpenClaw deployment guardrails.
- Do not unpublish normal superseded NPM versions; deprecate older versions with an upgrade message if needed.

Validation Summary

- Validated coverage includes OpenAI in-path AI Guardrails, out-of-path PII and HAPBlocker, Behavioral send_email blocking, Report Only allow behavior, and out-of-path fail-open / fail-close handling. Out-of-path blocked-topic prompt-stage behavior should be verified in the Radware portal for the selected template.



Operational Guidance

Key Notes For Customers

Diagnostics And Support

- Setup failures write a sanitized diagnostic log under `~/.openclaw/logs/radware-openclaw-setup-<timestamp>.log`. The log includes versions, selected mode, config path, attempted actions, plugin-install output, and the error stack.
- Out-of-path runtime diagnostics use `RADWARE_AGENTIC_DIAGNOSTIC_LOG` and default to `~/.openclaw/logs/radware-agentic-runtime.jsonl` when the setup-generated env file is loaded.
- Diagnostics include stage, endpoint host/path, model identifier, payload sizes, HTTP status, Event ID when present, and fail-open/fail-close decision. They do not include API keys, full prompts, tool arguments, or full payload bodies.
- Use clear portal identifiers such as `openclaw-in-path` and `openclaw-out-of-path` so events are easy to find in Radware.
- Report Only may not expose a client-visible Event ID when the action is allowed; verify those findings in the Radware portal.
- Use fail-close for production agents that can send, write, delete, or make network calls unless fail-open risk is explicitly accepted.

Where To Go Next

This document includes the basic installation journey. The GitHub repository and NPM package remain the source of truth for implementation details, current commands, validation scripts, diagnostics, and troubleshooting updates.

Implementation repository: <https://github.com/rdwr-seanr/openclaw-radware-agentic-protection>

NPM package: <https://www.npmjs.com/package/openclaw-radware-agentic-protection>